



INSTITUTO FEDERAL FARROUPILHA
***CAMPUS* FREDERICO WESTPHALEN**
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO



RELATÓRIO DO TRABALHO INTEGRADOR II
IFFARQL

Julia Francini Simon Barbosa
Milene Basso de Vargas
Robson Luan Theodoro

Frederico Westphalen
21 de setembro de 2026

1. INTRODUÇÃO

O trabalho apresentado a seguir foi o desenvolvimento de um Sistema Gerenciador de Banco de Dados simplificado, operado pelo terminal por meio da linguagem IFFARQL. O sistema interpreta comandos em português, organiza tabelas e registros, valida os dados durante as inserções, consultas e atualizações e exclusões.

A implementação foi realizada em Python, utilizando classes para modelar o banco de dados, as tabelas, as colunas, os registros e os tipos de dados. Cada tabela armazena seus registros em uma árvore binária de busca onde o ID é a chave. A aplicação ainda roda arquivos de comandos e armazena a estrutura do banco em JSON, o que possibilita recuperar os dados em uma nova sessão.

O sistema possui os comandos de manipulação e persistência previstos no enunciado. As seções seguintes apresentam as tecnologias, a organização da solução, um exemplo executável e as limitações da versão atual, incluindo diferenças conhecidas na validação de datas.

2. TECNOLOGIAS UTILIZADAS

O programa utiliza Python 3 e foi verificado com Python 3.12. Sua execução não depende de pacotes externos instalados com pip, pois as estruturas e operações do banco foram implementadas no próprio projeto.

As bibliotecas padrão utilizadas incluem *argparse* para os argumentos do terminal; *re* para reconhecer nomes e literais; *json* e *pathlib* para persistência e caminhos; os e *tempfile* para gravação em arquivo temporário e substituição do destino; *unicodedata* para remover acentos; *datetime* para operações com datas; e *math* para validar números finitos. Os testes também utilizam *subprocess*, *io* e *contextlib*.

3. ORGANIZAÇÃO E FUNCIONAMENTO

3.1. Interpretação dos comandos

O arquivo *main.py* mantém o terminal e apresenta os resultados e erros. O *tokenizador.py* separa os elementos do comando, preservando textos entre aspas e identificando valores inteiros,

decimais e booleanos. O *sintaxe.py* verifica a estrutura do comando. O *interpretador.py* coordena as validações, chama as operações e controla a confirmação das alterações.

As funções de *operacoes.py* criam e removem tabelas e inserem registros. O *consultas.py* consulta, atualiza e exclui registros, com apoio de *condicoes.py* para avaliar a condição ONDE. Os comandos exigem palavras reservadas em caixa alta; cada condição admite um comparador.

Terminal → *tokenizador.py* → *sintaxe.py* → *interpretador.py* → *operações do banco*

3.2. Estruturas e orientação a objetos

BancoDados organiza as tabelas por nome. Cada Tabela contém suas colunas, o contador *proximo_id* e uma *ArvoreRegistros*. Cada nó da árvore guarda o ID, o registro e as referências para os filhos. O percurso em profundidade em ordem produz os registros ordenados por ID. Embora listas auxiliares sejam usadas durante consultas e serialização, a estrutura que mantém os registros da tabela é a árvore.

A classe *TipoDado* define os métodos validar e operar. *Inteiro*, *Decimal*, *Booleano*, *Texto* e *Data* especializam esse comportamento. A chamada *coluna.tipo.operar(...)* escolhe em tempo de execução a implementação do tipo daquela coluna: números podem ser somados, textos concatenados e datas acrescidas de dias. Esse mecanismo demonstra polimorfismo dinâmico.

3.3. Integridade e persistência

O ID é criado automaticamente, não pode ser alterado pelo usuário e não é reutilizado após uma exclusão. O sistema verifica tipos, quantidade de valores, nomes e referências de chaves estrangeiras. Um registro referenciado não pode ser removido. Uma tabela só pode ser apagada quando está vazia e não é referenciada por outra tabela.

Cada comando de alteração é executado sobre uma cópia do banco. A cópia só substitui o banco em memória depois que as operações e o salvamento terminam com sucesso. Se ocorrer uma falha, a alteração não é confirmada. Esse procedimento implementa atomicidade por comando; em *CARREGARIFFARQL*, cada linha é uma transação independente.

A durabilidade utiliza salvamento automático em *JSON*. Antes de *SALVARBD* ou *CARREGARBD*, o destino é o cache; depois, o arquivo escolhido passa a ser o destino ativo. A gravação usa arquivo temporário, flush, fsync e os.replace. A recuperação reconstrói tabelas,

registros, tipos, referências e *proximo_id*. O cache deve ser carregado explicitamente. Não há controle de concorrência, conforme a dispensa de isolamento prevista no enunciado.

4. EXEMPLO DE ARQUIVO PARA CARREGAR IFFARQL

O arquivo `exemplos/comandos.txt`, incluído no projeto, contém o exemplo abaixo. Ele demonstra os cinco tipos de dados, relacionamento entre tabelas, atualizações, consultas, exclusão, continuidade dos IDs e salvamento. O arquivo utiliza UTF-8, com um comando completo por linha, sem numeração, comentários ou ponto e vírgula.

```
CRIATABELA cliente (nome TEXTO altura DECIMAL nascimento DATA tickets INTEIRO ativo BOOLEANO)
CRIATABELA pedido (descricao TEXTO idCliente INTEIRO CHAVESTRANGEIRA cliente)
INSERIREM cliente VALOR ("Adão" 1.80 "20/10/2010" 20 True)
INSERIREM cliente VALOR ("Maria Silva" 1.95 "15/05/2008" 10 False)
INSERIREM cliente VALOR ("Bruno" 1.70 "01/01/2000" 5 True)
INSERIREM pedido VALOR ("Primeiro pedido" 1)
ATUALIZATABELA cliente COM tickets = 30 ONDE id == 1
ATUALIZATABELA cliente COM tickets = tickets + 3 COM altura = altura - 0.15 ONDE nascimento ==
"15/05/2008"
ATUALIZATABELA cliente COM nome = nome + " Silva" ONDE id == 1
MOSTRADADOSDE cliente ONDE tickets >= 20
APAGADADOSDE cliente ONDE id == 3
INSERIREM cliente VALOR ("Carlos" 1.75 "01/01/2001" 8 True)
MOSTRADADOSDE cliente
MOSTRADADOSDE pedido
SALVARBD banco_exemplo.json
```

4.1. Execução do exemplo em Windows e macOS

No PowerShell, a partir da pasta principal do repositório, abra uma nova sessão:

```
cd Trabalho_2
python -B main.py --cache cache_exemplo.json
```

No prompt IFFARQL>, execute:

```
CARREGAR IFFARQL "exemplos/comandos.txt"
```

Ao final, cliente contém os IDs 1, 2 e 4: Adão é normalizado para Adao, Bruno é removido e Carlos recebe ID 4. O pedido continua ligado ao cliente 1. O arquivo *banco_exemplo.json* é salvo dentro de *exemplos*, pois caminhos relativos usados no TXT são resolvidos a partir da pasta desse arquivo.

Para conferir a persistência, digite SAIR, abra novamente o programa e execute, antes de criar tabelas:

```
CARREGARBD "exemplos/banco_exemplo.json"  
MOSTRADADOSDE cliente
```

5. LIMITAÇÕES CONHECIDAS

Datas. A implementação inclui suporte a anos bissextos, permitindo, por exemplo, a data 29/02/2028. Embora o enunciado tenha simplificado a validação considerando fevereiro com 28 dias, esse comportamento foi implementado como complemento à solução. Como limitação conhecida, a validação atual ainda aceita o ano 0000, enquanto o intervalo definido para o tipo DATA inicia em 0001. Além disso, datas com ano zero podem ser armazenadas, mas não são compatíveis com as operações realizadas por datetime.

Consultas e árvore. As consultas de mostrar_dados percorrem todos os registros, inclusive quando a condição é id == valor. A árvore possui busca direta por ID, mas essa otimização ainda não está integrada às consultas do terminal. A árvore também não possui balanceamento automático nas inserções e remoções.

Escalabilidade. Cada alteração copia, valida e salva o banco inteiro. Essa estratégia simplifica a confirmação atômica, mas aumenta o custo de memória, processamento e entrada e saída conforme a quantidade de dados cresce. O sistema é voltado a volumes didáticos.

Linguagem e valores. A linguagem admite somente uma condição ONDE, sem AND, OR, junções, agregações ou escolha de colunas na saída. Não há valores nulos, comandos em várias linhas ou escapes para aspas dentro dos textos. DECIMAL utiliza float, sujeito a aproximações de ponto flutuante.

Uso e concorrência. Não existem autenticação, autorização ou bloqueio de arquivo entre processos. Duas sessões escrevendo no mesmo arquivo podem sobrescrever alterações. O cache não é recuperado automaticamente na inicialização; o usuário deve usar CARREGARBD. A interface é exclusivamente de terminal.

6. VERIFICAÇÃO REALIZADA

Em 21/09/2026, os sete scripts de verificação existentes foram executados com Python 3.12 e terminaram sem falhas reportadas. Eles abrangem tokenização, sintaxe, interpretação, persistência, terminal e as operações da base. Os testes de integração verificam rollback, erros por linha, chaves estrangeiras, continuidade dos IDs e falhas simuladas de gravação.

Os resultados demonstram o comportamento coberto pelos scripts, sem significar conformidade integral com o enunciado. Verificações adicionais confirmaram as duas divergências de datas descritas acima. Para executar uma verificação de integração, na pasta *Trabalho_2*:

```
python -B -m testes.teste_interpretador  
python -B -m testes.teste_terminal
```